

touch pins and how to make an IoT application around it.

Materials to get started

The following materials are required to get started:

1. ESP32 module (check datasheet if using a different version)
2. USB Type-C cable to program ESP32 from a laptop or PC—most Android phones use this type of cable

3. LED with 220-ohm resistor for initial testing

4. Breadboard for mounting or placing components

5. Any metal plate (touchpad) for sensing touch (can use aluminium foil as touchpad by connecting a wire to it)

Software setup

To code ESP32, Arduino integrated development environment (IDE) is

needed. Arduino IDE is a cross-platform application, written in Java and coded in C/C++ with some special rules. Download the latest Arduino IDE from www.arduino.cc/en/Main/Software

Arduino IDE does not contain support for ESP32 family. To install ESP32 board library in Arduino IDE, scan the QR code given in Fig. 15(a).

Understanding the source code. Before uploading the code to ESP32, make changes in the code as shown in Fig. 6.

< WiFi.h > library contains all Wi-Fi functions used in the code.

Replace your Wi-Fi credentials in the code given below, within double quotes.

```
const char* ssid = "xxxx";
const char* password = "xxxx";
```

Make global declarations at the beginning of the code shown in Fig. 6.

In Void Setup() shown in Fig. 7, set baud rate at 115200 (default speed). Set outputs and initialise Wi-Fi to connect it once. All commands under Void Setup() run only once after every reset.

In void loop() shown in Fig. 8, place the main code that needs to run repeatedly.

It is possible to directly read touch GPIOs using touchRead() function. Save it to any variable. (Here, it is

```
void setup()
{
  Serial.begin(115200);
  pinMode(output5, OUTPUT);
  digitalWrite(output5, LOW);
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  server.begin();
}
```

Fig. 7: Void Setup()

```
void loop()
{
  WiFiClient client = server.available();
  s1=touchRead(T0);
  if (s1<90)
  {
    state=!state;
    Serial.print("State: ");
    Serial.println(state);
    delay(500);
    digitalWrite(output5, state);
  }
  if (client)
  {
    Serial.println("New Client.");
    String currentLine = "";
    while (client.connected())
    {
      if (client.available())
      {
        char c = client.read();
        Serial.write(c);
        header += c;
        if (c == '\n')
        {
          if (currentLine.length() == 0)
          {
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
          }
        }
      }
    }
  }
}
```

Fig. 8: Void loop()

```
digitalWrite(output5, state);
if (state==true)
{
  Serial.println("GPIO 5 on");
  output5State = "on";
}
else if (state==false)
{
  Serial.println("GPIO 5 off");
  output5State = "off";
}
client.println("<!DOCTYPE html><html>");
client.println("<head><meta name='viewport' content='width=device-width, initial-scale=1'>");
client.println("<link rel='icon' href='data:,'>");
client.println("<style>html { font-family: Helvetica; display: inline-block; margin: 0px auto; text-align: center; }");
client.println("<body> { background-color: #191919; border: none; color: white; padding: 16px 40px; }");
client.println("<div> { text-align: center; font-size: 30px; margin: 2px; cursor: pointer; }");
client.println("<div> { background-color: #7f7f7f; padding: 5px; }");
client.println("<div> { background-color: #7f7f7f; padding: 5px; }");
client.println("<div> { background-color: #7f7f7f; padding: 5px; }");
if (output5State=="on")
{
  client.println("<p><a href='\"/\"'><button class='\"button\"'>ON</button></a></p>");
}
else
{
  client.println("<p><a href='\"/\"'><button class='\"button\"'>OFF</button></a></p>");
}
```

Fig. 9: HTML page in code

ESP32 with Touch

Light is On

ON

Fig. 10: HTML Web page on browser

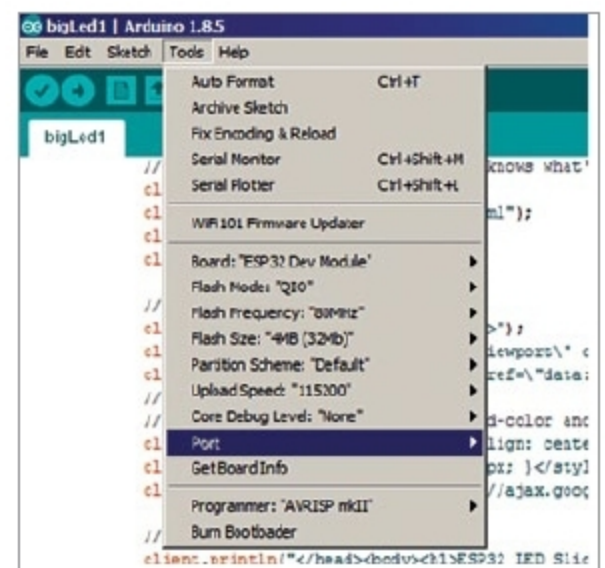


Fig. 11: Select COM port